



ALII - API for Lab Item Interoperability

Version - 1.9

PEARSON VUE PROPRIETARY

Copyrighted Material of Pearson Education, Inc. or its affiliate(s).

Created 2018. All rights reserved.

REVISION HISTORY

Changes to the text of this document are indicated by bars in the outside margin adjacent to the affected text.

Date	Change Description
April 18 2018	Draft
June 10 2018	v1.0 distributed for review
June 25 2018	v1.1 distributed for review. Added security section to the document.
July 20 2018	v1.2 added the revised Lab Error Callback (section 4.4) s.b.
August 13 2018	v1.3 updated sample response parameters in sections 4.1 and 4.2 to match response table specification. resequenced Section 4. 4.3 was repeated twice. Added clientID to Lab Error Callback
October 8, 2018	Changes to scoring, remove grade, add score at response level.
November 26, 2018	V1.4 added 4.6 client side communication. Added ErrorUrl to initialization parameters.
December 3, 2018	Added instance_id to both requests in 4.6.
February 14, 2019	v1.5 Added perf object to “status” message in 4.6.2
April 17, 2019	v1.6 added 4.6.3 Type Text client side api.
May 23rd, 2019	v1.7 added expirationTime and diagnostics parameters to /labInstance (in section 4.1). Added section 4.7 Lab Activity.
June 12, 2019	v1.8. split deliveryRegion, dataCenter, and startupDuration into separate response parameters. Minor typo corrections.
July 26, 2019	v1.9. Moved diagnostics (expirationTime, deliveryRegion, dataCenter, and startupDuration) to response table where they

	were originally intended. Removed refresh from /score Added more info to /score description Minor note updates for fields.
--	---

Table of contents

1. Introduction	4
2. Glossary	5
3. Scope	6
4. Exam Delivery	6
4. API Specifications	9
4.1 Initialize Lab Item	9
4.2 Get Lab Item URL	14
4.3 Get Lab Item Score	17
4.4 Archive Lab Item Instance	22
4.5 Lab Error Callback	23
4.6 Client-side communication	26
4.6.1 Status Check - driver to lab	26
4.6.2 Status return - lab to driver	26
4.6.3 Type Text - driver to lab	28
4.7 Lab activity	29
5. Lab instance status	29
6. Security	31
6.1 Transport	31
6.2 Authentication	31

1. Introduction

This document describes the APIs for supporting PBT (Performance Based Testing) exams in Pearson VUE. It describes in detail the web service APIs to integrate a lab partner with any Internet Based Testing (IBT) platform used in Pearson VUE.

2. Glossary

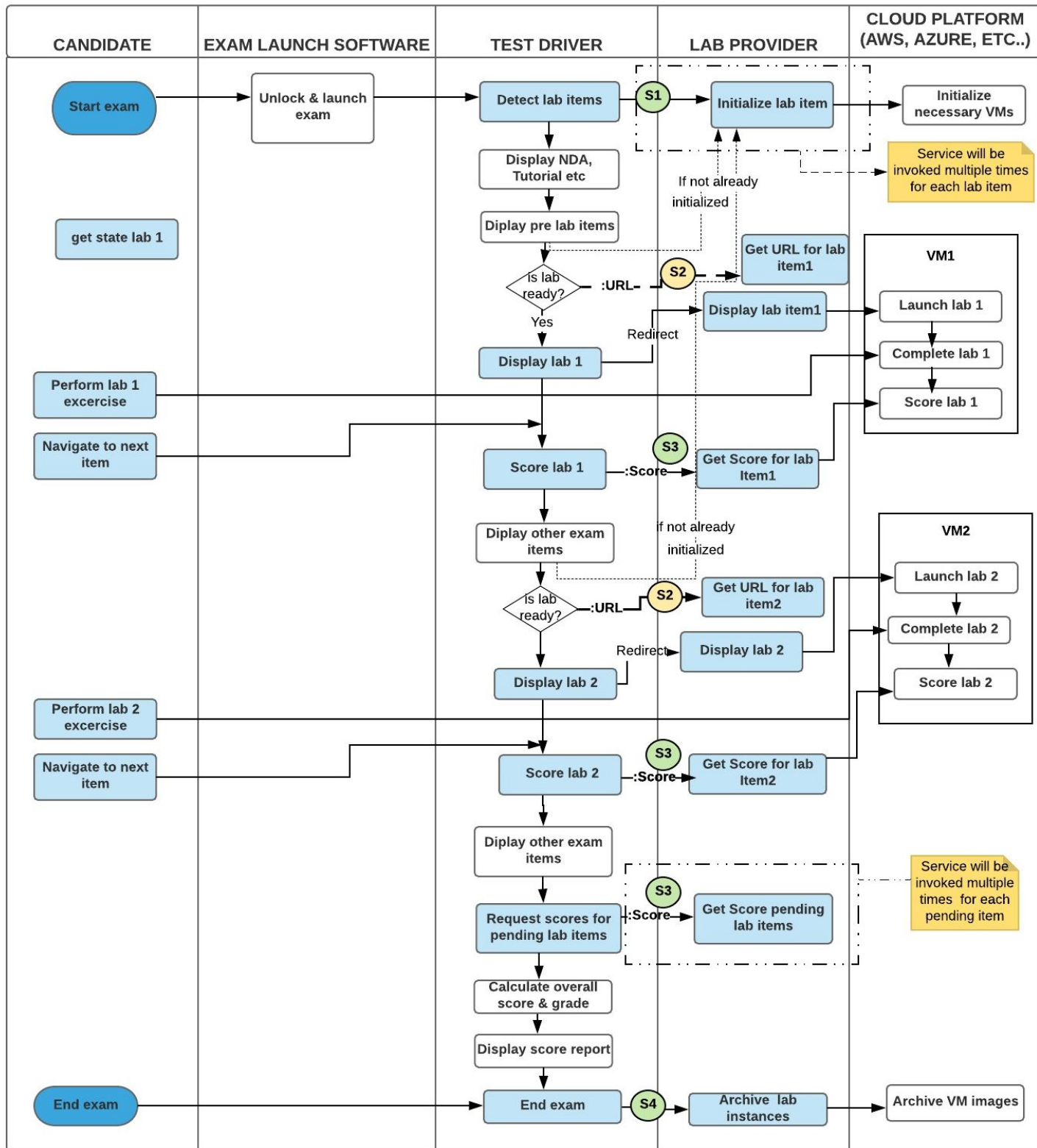
Term	Definition
PBT exams	Performance Based Testing exams
Candidate	End user taking exam
Lab partner	Partner who is responsible for hosting and managing virtual labs
IBT	Internet Based Testing
VUE	Shorthand for Pearson VUE, testing division of Pearson
ITS	Internet Testing Systems
Client	Pearson VUE test sponsor

3. Scope

The scope of this document includes the API integrations required to successfully deliver and score a lab item for an IBT exam. API integration required for publishing a lab item into an item authoring platform is considered out of scope for this document.

4. Exam Delivery

It is reasonable to assume that an exam contains multiple lab items. The below diagram explains the sequence of steps involved in presenting and scoring lab items during an exam delivery.



PEARSON VUE PROPRIETARY

Copyrighted Material of Pearson Education, Inc. or its affiliate(s).

Created 2018. All rights reserved.

1. There may be several steps involved in launching a lab item like initializing a VM instance, starting any custom software needed to deliver the lab item etc. These steps may take several seconds to minutes based on the cloud platform and the complexity of the lab items. It is important to reduce candidate wait times for these lab items. Either at the launch of the exam or at the beginning of a section, the IBT test driver will make its best possible effort to identify the lab items that will be presented to the candidate. It will invoke the “**S1-InitializeLabItem**” API and provides the lab item identifiers to the lab partner. Lab partner is expected to return lab instance id and if possible a URL for each lab item instance. This allows the lab partner to initialize the necessary VM and the lab software required for launching each lab item.
2. To further reduce candidate wait times, test publishers will be advised to design the exams to have standard exam items at the beginning of the exam or a particular section and keep the lab items to the end if possible.
3. When the test driver is ready to present the lab item to the candidate, test driver will check if the lab item is already initialized and a URL is available. If a URL is not returned in the above step, test driver will invoke the “**S2-GetLabItemURL**” API. In this API call, test driver will provide the lab item instance id. Lab partner is expected to return a URL for the lab item.
If the lab item id is not already initialized, lab partner can return an instance id , a status of “pending” and expected wait item in seconds. Test driver will display appropriate message to the candidate and invoke the API again after the expected wait time. At this step, it is possible that the test driver did not detect the lab item at the beginning of the exam or the section and thereby did not invoke the “S1-InitializeLabItem API”. In that case, test driver will invoke “S1-InitializeLabItem API” before invoking “S2-GetLabItemURL”.
4. On getting a URL from the lab partner, test driver will create an iFrame and redirect the candidate to the URL. Lab partner will display the lab item to the candidate.
5. Candidate will be presented with instructions to perform different tasks on the lab.
6. After successfully completing the lab item, candidate will click the appropriate navigation link on the test driver to move to the next item in the exam.
7. Test driver invokes “**S3-GetLabItemScore**” API, to retrieve the response and the score for the lab item. If the lab partner needs more time to provide the response and score for the lab item, a status of “pending” and an expected wait time will be returned back in the response.
8. Test driver will proceed with displaying the next item in the exam. However, it is possible for an exam to be designed for the test driver to wait for the lab item response and score. It may be required for the test driver to select the next item in the exam.
9. At the end of the exam, Test driver will check if there are any lab items with scores pending and invokes “S3-GetLabItemScore” API to retrieve the response and score for each pending lab item. At this point, test driver will wait until all responses and scores are retrieved from the lab partner.
10. Test driver calculates the overall score, grade and displays a score report to the candidate.
11. When the candidate terminates the exam, test driver invokes “**S4-ArchiveLabItemInstance**” API for each lab item. Lab partner can safely assume that the exam session is terminated and the lab instance will be no longer needed. Lab partner is expected to archive the vm instances and store them.

4. API Specifications

All APIs will be implemented as **REST** APIs and all responses are formatted as **JSON**.

4.1 Initialize Lab Item

POST /labInstance

This API is required for the test driver to notify the lab partner at the beginning of the exam to initialize the lab items that could be potentially delivered to the candidate during the exam.

Request Parameters

Name	Description	Data type	Required
clientId	Client Id. For Pearson VUE internal purpose, client code will be used as client id. Lab provider may need this information for routing the lab request to different API gateways or cloud platforms etc.	String	Yes
labId	lab id. GUID for a lab item in an exam. This should be unique across different test delivery programs and exams.	String	Yes
candidateId	Id of the test taker. This may be used by the lab provider for linking a lab to a specific user account	String	Yes

	or other auditing purpose.		
registrationId	Exam Registration ID. Unique identifier for each exam delivery event.	String	Yes
ipAddress	IP address from which the candidate is taking exam.	String	No
country	Candidate country of residence. 3 character ISO country code.	String	No
errorUrl	URL of test driver /error endpoint if not a predetermined value provided by test driver platform ahead of time.	String	No

Response parameter

Name	Description	Data type	Required
labInstanceId	Lab item instance id	String	Yes, If status is “Initialized” or “Pending”
url	URL for the lab item instance	String	No
status	Status of the lab item instance	Enumeration. Possible values are “PENDING_INITIALIZATION”, “INITIALIZED”, “ERROR” Refer section 5 “Lab Instance Status” for the	Yes

		complete list of lab item status.	
waitTime Note: It is up to the test driver implementation to honor the wait time.	Wait time for the lab item instance (in seconds)	int	Yes, if status is “PENDING”
custom	Array of name value pairs. This provides a mechanism for lab providers to return custom data that needs to be displayed by the test driver to the candidate for the lab item. For example, this could be username and password for the lab that the candidate has to enter.	Array	No
errorCode	Error code	String	Yes, if status is “ERROR”
errorMessage	Error message	String	Yes, if status is “ERROR”. Free form detailed error message generated by the lab provider.
expirationTime	Diagnostic info - Date time that the lab will expire. 1/1/1970 if no expiration. null/omitted if not applicable.	Date time	No
diagnostics	Array of name value pairs. This provides a mechanism for lab providers to return	Array	No

	diagnostic information.		
deliveryRegion	Lab provider delivery region, freeform text. Example: us-east-1	String	No
dataCenter	Lab provider delivery data center, freeform text. Example: New Jersey	String	No
startupDuration	Time in seconds lab instance took to start up.	int	No

Sample Request

```
{
  "clientId": "clientCode",
  "labId": "1001",
  "candidateId": "Cand1325376800",
  "registrationId": "2325376900",
  "ipAddress": "69.89.31.226",
  "country": "USA"
}
```

Sample Response - Success

```
{
  "labInstanceId": "1234",
  "url": "https://labtesting.com/laburl?id=1234",
  "status": "INITIALIZED",
  "custom": [
    { "name": "userName", "value": "ALII_user" },
    { "name": "password", "value": "ALII_Pwd" }
  ]
}
```

Sample Response - Pending with wait time

```
{
  "labInstanceId": "1234",
  "url": "https://labtesting.com/laburl?id=1234",
  "status": "PENDING_INITIALIZATION",
  "waitTime": 60
}
```

Sample Response - Failure

```
{
  "status": "ERROR",
  "errorCode": "INVALID_LAB_ID",
  "errorMessage": "Invalid lab identifier"
}
```

Error Codes

Code	HTTP Status	Description
INVALID_LAB_ID	400	Invalid lab identifier
INVALID_IP_FORMAT	400	Invalid ip address format
INVALID_COUNTRY_CODE	400	Invalid country code
INVALID_CLIENT_ID	400	Invalid client identifier
INVALID_CANDIDATE_ID	400	Invalid candidate identifier

INVALID_REG_ID	400	Invalid registration identifier
SYSTEM_ERROR	500	Unexpected system error. Eligible for auto recovery.

4.2 Get Lab Item URL

GET /labInstance/{id}

This API is required for the test driver to retrieve the URL for a lab instance.

Request Parameters

Name	Description	Data type	Required
id	Lab instance id	String	Yes

Response parameter

Name	Description	Data type	Required
labInstanceId	Lab instance id	String	Yes
url	URL for the lab item instance	String	No
status	Status of the lab item instance	Enumeration. For list of possible values refer section 5 “Lab Instance Status” for the complete list of lab item status.	Yes
waitTime Note: It is up to the test driver	Wait time for the lab item instance (in seconds)	int	Yes if status is “PENDING”

implementation to honor the wait time.			
custom	Array of name value pairs. This provides a mechanism for lab providers to return custom data that needs to be displayed by the test driver to the candidate for the lab item. For example, this could be username and password for the lab that the candidate has to enter.	Array	No
errorCode	Error code	String	Yes, if status is “ERROR”
errorMessage	Error message	String	Yes, if status is “ERROR”. Free form detailed error message generated by the lab provider.

Sample Request

<https://labtesting.com/api/1234>

Sample Response - Success

```
{
  "labInstanceId": "1234",
  "url": "https://labtesting.com/laburl?id=1234",
  "status": "INITIALIZED",
  "custom": [
    { "name": "userName", "value": "ALII_user" },
    { "name": "password", "value": "ALII_Pwd" }
  ]
}
```


Sample Response - Pending with wait time

```
{
  "labInstanceId": "1234",
  "url": "https://labtesting.com/laburl?id=1234",
  "status": "PENDING_INITIALIZATION",
  "waitTime": 60
}
```

Sample Response - Failure

```
{
  "status": "ERROR",
  "errorCode": "INVALID_LAB_ID",
  "errorMessage": "Invalid lab identifier"
}
```

Error Codes

Code	HTTP Status	Description
INVALID_LAB_ID	400	Invalid lab identifier
INVALID_IP_FORMAT	400	Invalid ip address format
INVALID_COUNTRY_CODE	400	Invalid country code
INVALID_CLIENT_ID	400	Invalid client identifier
INVALID_CANDIDATE_ID	400	Invalid candidate identifier
INVALID_REG_ID	400	Invalid registration identifier
SYSTEM_ERROR	500	Unexpected system error. Eligible for auto recovery.

4.3 Get Lab Item Score

GET /labInstance/{id}/score

This API is required for the test driver to retrieve the response and score for a lab item. If called more than once lab provider should provide a refreshed score if applicable.

NOTE! Please work with your testing platform provider to understand the options for how responses and other scoring information will be represented in delivery results. Data processing will vary per provider.

Request Parameters

Name	Description	Data type	Required
labInstanceId	lab instance id	String	Yes

Response parameter

Name	Description	Data type	Required
status	Status of the score.	Enumeration "COMPLETE", "PENDING", "ERROR"	Yes
waitTime Note: It is up to the test driver implementation to honor the wait time.	Wait time for the lab item instance (in seconds).	Integer	Yes, if score status is "PENDING"
errorCode	Error code.	String	Yes, if status is "ERROR"
errorMessage	Error message.	String	Yes, if status is "ERROR". Free form detailed error message

			generated by the lab provider.
Item Data			
itemStatus	The status field exposes details about the item's state that are important to processing by the test delivery platform. Refer to the “Item Status” table below.	Enumeration “COMPLETE”, “INCOMPLETE”, “NOT_PRESENTED”, “SKIPPED”	Yes
isCorrect	Indicates the correctness of the item.	Boolean “true”, “false”	Yes, if Status (of responding) is “COMPLETE”
score	Item score. See note above.	Decimal	No
responses	A response is the outcome of a single candidate interaction with a task/question posed by the lab item. The response format is required to be an array (with a minimum of 1 element).	Array of response data. The response data format is defined in the table below “Response Data”	Yes, if Status (of responding) is COMPLETE
Response Data			
id	The response identifier that must be unique within the responses array and within the entire test module. <u>id rules:</u> You can use any combination of the following characters in the id, and spaces are not allowed: • Uppercase English letters: (A–Z) • Lowercase English letters: (a–z) • Westernized Arabic numerals: (0–9) • Symbols: Period (.), Colon (:),	String	Yes

	Dash (-), Underscore (_) - ident string must not be > 36 chars - cannot have duplicate response ids within a lab item.		
score	score of the response, see note above	Decimal	No
value	The value field can be any opaque data, so long as it is a single, valid string. This means that there cannot be any unescaped quotes or other reserved characters. Lab providers can use a function such as <code>JSON.stringify()</code> to automatically escape quotes within the JavaScript code and <code>JSON.parse()</code> to later undo the process.	String	Yes
state	The state of the item response. The value should be a base64 encoded string. This information will be used for downstream item analysis. The size should be limited to a max of 4k .	String	No
isCorrect	Indicates the correctness of the response.	Boolean “true”, “false”	Yes, If Status is “COMPLETE”
responseStatus	Status of the item response. Refer to the “Response Status” table below.	Enumeration “COMPLETE”, “INCOMPLETE”, “SKIPPED”	Yes
variables	may return a variables block as defined below		No
Variables			
variables	Array of name value pairs. A lab	Array	No

	item may return a set of scoring and other additional item variables.		
name	Name of the variable.	String	Yes
type	Data type of the variable.	Enumeration INTEGER, DECIMAL, BOOLEAN, STRING	Yes
value	value of the variable.	String	Yes

Sample Request

<https://labtesting.com/api/1234/score>

Sample Response - Success

```
{
  "status": "COMPLETE",
  "itemStatus": "COMPLETE",
  "isCorrect": "true",
  "score": 90.2,
  "responses": [
    {
      "id": "1234",
      "score": 90.2,
      "value": "some data",
      "state": "some data",
      "isCorrect": "true",
      "responseStatus": "COMPLETE",
      "variables": [
        {
          "name": "name1",
          "type": "string",
          "value": "val1"
        },
        ...
      ]
    },
    ...
  ],
  "variables": [
    {
      "name": "name1",
      "type": "string",
      "value": "val1"
    },
    ...
  ]
}
```

Sample Response - Failure

```
{
  "status": "ERROR",
  "errorCode": "INVALID_LAB_ID",
  ...
}
```

```
    "errorMessage": "Invalid lab instance identifier"
  }
```

Item Status

Response Status	Description
COMPLETE	The candidate completed the entire sequence of interactions defined for the lab item.
INCOMPLETE	The lab item was presented, but the candidate did not complete the entire sequence of interactions defined for the lab item.
NOT_PRESENTED	Lab item was never presented to the candidate. NB: Cannot be set by the lab provider, this is only settable by the driver.
SKIPPED	Lab item presented to the candidate, but the candidate skipped the lab item

Response Status

Response Status	Description
COMPLETE	The candidate completed the entire sequence of interactions for a task/question defined within the lab item.
INCOMPLETE	The candidate did not complete the entire sequence of interactions for a task/question defined within the lab item.
SKIPPED	Task/question presented to the candidate, but the candidate skipped it.

Error Codes

Code	HTTP Status	Description
INVALID_LAB_ID	404	Invalid lab identifier
ERROR_SCORING	500	Unable to score the lab item.
SYSTEM_ERROR	500	Unexpected system error. May be eligible for auto recovery.

4.4 Archive Lab Item Instance

POST /labInstance/{id}/archive

This API is required for the test driver to archive lab item instance id.

Request Parameters

Name	Description	Data type	Required
id	lab instance id	String	Yes

Response parameter

Name	Description	Data type	Required
status	Status of the lab item instance	Enumeration “Archived”, “Error”	Yes, on successful archival
errorCode	Error code	String	Yes, on error
errorMessage	Error message	String	Yes, on error. Free form detailed error message generated by the lab provider.

Sample Request

```
https://labtesting.com/api/1234/archive
```

Sample Response - Success

```
{
  "id": "1234",
  "status": "Archived"}
```

Sample Response - Failure

```
{
  "errorCode": "INVALID_LAB_ID",
  "errorMessage": "Invalid lab instance identifier"
}
```

Error Codes

Code	HTTP Status	Description
INVALID_LAB_ID	404	Invalid lab identifier
SYSTEM_ERROR	500	Unexpected system error. May be eligible for auto recovery.

4.5 Lab Error Callback

POST /labError

This API is required for the lab partner to notify the test driver while in the middle of a lab item being delivered that an error or some other status change has occurred in that lab item. This API will be hosted by the test delivery platform.

Lab Error Callback Parameters

Name	Description	Data type	Required
------	-------------	-----------	----------

PEARSON VUE PROPRIETARY

Copyrighted Material of Pearson Education, Inc. or its affiliate(s).

Created 2018. All rights reserved.

clientId	Client Id. The lab provider should pass the client Id supplied in the Initialize Lab Item call.		
labInstanceId	Lab item instance id	String	Yes
recoverable	Status of the lab item instance	true/false	Yes
waitTime Note: It is up to the test driver implementation to honor the wait time.	Wait time for the recovered lab item instance to be available (in seconds)	int	Yes, if recoverable is true
errorMessage	Error message	String	Yes, Free form detailed error message generated by the lab provider.

Sample Error Callback - Terminal Error

```
{
  "clientId": "clientCode",
  "labInstanceId": "1234",
  "recoverable": "false",
  "errorMessage": "We have encountered an error and the lab item cannot be delivered."
}
```

Sample Error Callback - Recoverable Error with wait time

```
{
  "clientId": "clientCode",
  "labInstanceId": "1234",
  "recoverable": "false",
  "waitTime": 60,
  "errorMessage": "We have encountered an error and are working to bring the lab item back."
}
```

Sample Response - Acknowledged

```
{
  "labInstanceId": "1234",
  "status": "ACK"
}
```

Sample Response - Failure

```
{
  "labInstanceId": "1234",
  "status": "ERROR",
  "errorCode": "INVALID_LAB_ID",
}
```

Recoverable States

Recoverable States	Description
False	The test driver will notify the user that the item has failed
True	The test driver waits the specified “waitTime” and calls “GET/labInstance/{id}” which will return the URL to be reconnected to

Response Error Codes

Code	HTTP Status	Description
INVALID_LAB_ID	404	Invalid lab identifier
INVALID_SCORE_REPORT_URL	500	The Score Report URL was not valid
SYSTEM_ERROR	500	Unexpected system error.

4.6 Client-side communication

Client-side communication via the [Window.postMessage\(\)](#) API provides a mechanism for the front end of the driver and lab to communicate directly. This API is used to provide improved candidate experience with timing and lab responsiveness.

4.6.1 Status Check - driver to lab

statusCheck

This API is required for the test driver to retrieve the status of the lab. Not to be used as a heartbeat.

Variables in Message data object

Name	Description	Data type	Required
event_id	Required to be "statusCheck"	String	Yes
instance_id	lab instance ID, can be verified by lab.	String	Yes

Example

```
var receiver = document.getElementById('lab_frame').contentWindow;

receiver.postMessage({
  event_id: 'statusCheck',
  instance_id: '12345'
},
"*");
```

4.6.2 Status return - lab to driver

status

This API is required for the test driver to retrieve the status of the lab. The lab will initiate sending a status message on every status change. The lab will send this response when queried with a statusCheck message. The lab will return the lab instance ID in the request. This is so if there is more

than one iFrame loaded at once the driver can determine where the message came from. The lab can choose to validate the id sent in the 'statusCheck' message or return the one it received.

Variables in Message data object

Name	Description	Data type	Required
event_id	Postmessage event_id, required to be "status"	String	Yes, by postMessage
value	Value will be set to one of the statuses in the below table.	String	Yes
instance_id	lab instance ID, used by driver to determine where message came from.	String	Yes
Performance Data			
perf	An object containing performance metrics for the lab. Currently, only latency is supported by this API.	Object	No
latency	The delay between a lab provider's request and a lab's response measured in seconds.	Integer	No

Example

```
window.parent.postMessage({
  event_id: 'status',
  value: 'loaded',
  instance_id: '12345',
  perf: {latency: 10}
},
"*");
```

Status Values

Note: Case insensitive

Code	Description
loading	Lab is in the process of loading
loaded	Lab has loaded. Driver will unpause clock upon receipt.
error	Lab has encountered an error. Driver will pause clock upon receipt.
reconnected	Lab has reconnected. Driver will unpause clock upon receipt.

4.6.3 Type Text - driver to lab

typeText

This API is used to send text messages from the driver to the lab. The text is typed into the lab at the cursor location.

Variables in Message data object

Name	Description	Data type	Required
event_id	Required to be “typeText”	String	Yes
value	The text that will be typed into the lab.	String	Yes

Example

```
var receiver = document.getElementById('lab_frame').contentWindow;
```

```
receiver.postMessage({
  event_id: 'typeText',
  value: 'Hello'
},
"*");
```

4.7 Lab activity

GET /labInstance/{id}/activityLog

This API is required for the test driver to retrieve activity details from the lab provider.

Request Parameters

Name	Description	Data type	Required
id	Lab instance id	String	Yes

Response parameter

Name	Description	Data type	Required
activityLog	Activity that has occurred with the lab provider during the session. Format is preferred to be time stamped log statements with quoted key-value pairs so it can be indexed and searched by logging software: 2019-05-15 14:36:34.148+0000 [INFO] user clicked button: registrationID=12345, labInstance=12345687, buttonName="testButton"	String, size limit 25MB	No

5. Lab instance status

There are five possible states for a lab instance and the below table explains them in detail

Status	Description
PENDING_INITIALIZATION	Lab instance is in the process of being initialized.
INITIALIZED	Lab instance is initialized and is ready for use.
DELIVERED	Lab item successfully delivered and is ` by the lab provider.

ERROR	Unable to initialize lab item.
ARCHIVED	Archived lab instance.

6. Security

6.1 Transport

All API communications will be over HTTPS limited to strong cipher suites. TLS v1.1 or higher is required.

6.2 Authentication

All ALII APIs must be protected using an API key. The API key must be passed in the “Authorization” HTTP header.

Authorization: <API KEY>

Example:

Authorization: 1234567890123456789

The party hosting the APIs is responsible for generating the key and distributing it securely to the caller in advance. The mechanics of generating and distributing the API keys is out of scope for the ALII specification and it is up to the party hosting the APIs. Key rotation policies is also out of scope for the ALII specification and it is up to the party hosting the APIs